Day 6

Class:

It is a collection of objects. **Classes** are the ***blueprints*** for creating objects. It defines a set of attributes and methods that the created objects have.

- They are created using "**class**" keyword
- Attributes are the variables that belong to a class
- Attributes are always public and can be accessed using "." Operator

```python
class Dog:
    species = "Canine"  # Class attribute

    def __init__(self, name, age):
        self.name = name  # Instance attribute
        self.age = age  # Instance attribute
```

Object:

It is an instance of a class. It represents a specific implementation of the class and holds its own data.

It consists of -

- State: represented by the attribute and reflects the properties of an object
- Behaviour: represented by the methods of an object
- Identity: gives a unique name to an object and enables one object to interact with other objects

```python
class Dog:
    species = "Canine"  # Class attribute

    def __init__(self, name, age):
        self.name = name  # Instance attribute
        self.age = age  # Instance attribute

# Creating an object of the Dog class
dog1 = Dog("Buddy", 3)
print(dog1.name)
print(dog1.species)
```

Output

```
Buddy
Canine
```

Class attributes:
They belong to the class itself, and will be shared by all the instances. These are defined in the class body parts at the top.

```
# Write Python code here
class sampleclass:
    count = 0     # class attribute

    def increase(self):
        sampleclass.count += 1

# Calling increase() on an object
s1 = sampleclass()
s1.increase()
print(s1.count)

# Calling increase on one more
# object
s2 = sampleclass()
s2.increase()
print(s2.count)

print(sampleclass.count)
```

Output:

```
1
2
2
```

Instance attributes:

These are not shared by objects. Every object has its own copy of the instance attribute.

To list the attributes of an instance/object, there are 2 functions -

- **vars()** —> it displays the attribute of an instance in the form of a dictionary.
- **dir()** —> it displays more attributes than 'vars()' function, as it displays the class attributes as well.

```python
# Python program to demonstrate
# instance attributes.
class emp:
    def __init__(self):
        self.name = 'xyz'
        self.salary = 4000

    def show(self):
        print(self.name)
        print(self.salary)

e1 = emp()
print("Dictionary form :", vars(e1))
print(dir(e1))
```

Output :

```
Dictionary form :{'salary': 4000, 'name': 'xyz'}
['__doc__', '__init__', '__module__', 'name', 'salary', 'show']
```

__init__() method:

It is a constructor which is automatically called when a new object of a class is created. Its main purpose is to initialise the object's attributes and set up its initial state.

```python
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

a = Person("Max", 30)
b = Person("Emilia", 25)

print(a.name, a.age)
print(b.name, b.age)
```

Output

```
Max 30
Emilia 25
```

```python
class Dog:
    def __init__(self, name, breed="Mixed", age=1):
        self.name = name
        self.breed = breed
        self.age = age

a = Dog("Buddy")
b = Dog("Max", "Golden Retriever", 5)

print(a.name, a.breed, a.age)
print(b.name, b.breed, b.age)
```

Output

```
Buddy Mixed 1
Max Golden Retriever 5
```

Class method: @classmethod

It is used to create a method that works with the class instead of object. It is used to access class variables, and perform operations related to the class.

```python
class Car:
    brand = "Toyota"

    @classmethod
    def show_brand(cls):
        print(cls.brand)

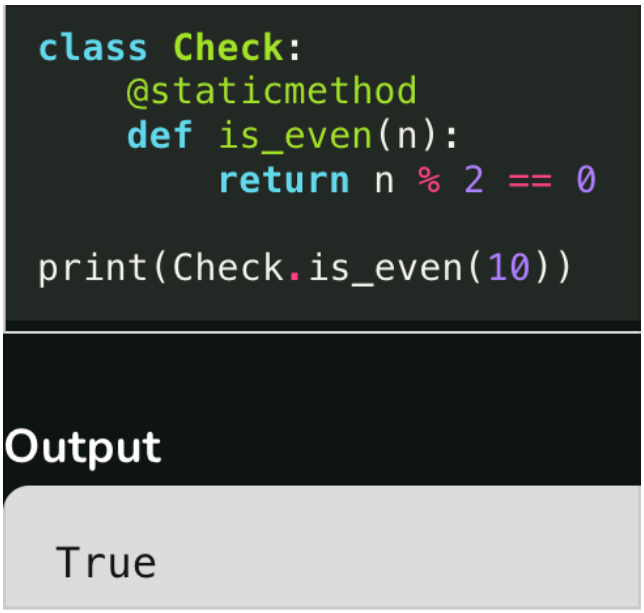
Car.show_brand()
```

Output

```
Toyota
```

Static method: @staticmethod

It is a method defined inside a class that does not depend on any instance or class data. It is used when a function logically belongs to a class but does not need access to self



Instance method:

They are methods associated with instances of a class and operate on the instance's data. When defining an instance method, the method's first parameter is typically named self, which refers to the instance calling the method.

```
class MyClass:
  def instance_method(self, arg1, arg2, ...):
    # Instance method logic here
    pass
```

Encapsulation:
The idea is to bind the data members and methods into a single unit.

- Helps in better maintainability, readability and usability
- Protects data from unauthorised access
- Enhances modularity by hiding internal implementation details
- Reflects real-world scenarios like restricting direct access to a bank account balance

```python
class Employee:
    def __init__(self, name, salary):
        self.name = name           # public attribute
        self.__salary = salary     # private attribute

emp = Employee("Fedrick", 50000)
print(emp.name)
print(emp.__salary)
```

```
Fedrick

ERROR!
Traceback (most recent call last):
  File "<main.py>", line 8, in <module>
AttributeError: 'Employee' object has no attribute '__salary'
```

Here, '***self.__salary = salary***' is a **private** attribute and cannot be accessed directly

Interface:

It is a collection of method signatures that should be provided by the implementing class. Implementing an interface is a way of writing an organised code and achieve abstraction. The package '**zope.interface'** provides an implementation of "object interfaces" for Python
Interface acts as a blueprint for designing classes, so interfaces are implemented using **implementer** decorator on class. If a class implements an interface, then the instances of the class provide the interface.

```python
import zope.interface


class MyInterface(zope.interface.Interface):
    x = zope.interface.Attribute("foo")
    def method1(self, x):
        pass
    def method2(self):
        pass

@zope.interface.implementer(MyInterface)
class MyClass:
    def method1(self, x):
        return x**2
    def method2(self):
        return "foo"
```

Inheritance:
It allows the child class to inherit attributes and methods of the parent class.

```python
class Animal:
    def __init__(self, name):
        self.name = name

    def info(self):
        print("Animal name:", self.name)

class Dog(Animal):
    def sound(self):
        print(self.name, "barks")

d = Dog("Buddy")
# Inherited method
d.info()
d.sound()
```

Output

```
Animal name: Buddy
Buddy barks
```

Method overriding:

Method overriding occurs when a child class defines a method with the same name as method in its parent class. This allows the child class to provide its own implementation while retaining the inheritance relationship.

```python
class Animal:
    def display(self):
        print("This is an animal")

class Dog(Animal):
    def display(self):
        print("This is a dog")

obj = Dog()
obj.display()
```

Output

```
This is a dog
```

- Animal class contains display(). Dog class inherits from Animal class and defines its own display().
- Since the child class has a method with the same name, it overrides the parent class method
- When 'obj.display()' is called, Python executes the version defined in the Dog class instead of the one in the Animal class.

Polymorphism

Polymorphism means "many forms" and allows the same method, function or operator to behave differently depending on the object or data it works with.
Eg : an online payment system, different payment methods such as Credit Card, UPI, Wallet and NetBanking all use the same 'processPayment()" method

- The method name remains the same, but each payment type performs its own specific payment process.

```python
class Animal:
    def sound(self):
        return "Some generic sound"

class Dog(Animal):
    def sound(self):
        return "Bark"

class Cat(Animal):
    def sound(self):
        return "Meow"

animals = [Dog(), Cat(), Animal()]
for animal in animals:
    print(animal.sound())
```

Output

```
Bark
Meow
Some generic sound
```

Composition v/s Inheritance:

Composition	Inheritance
Is-a relationship	Has-a relationship
Inherits class behaviour	Uses another object
Child depends on parent hierarchy	Container depends on contained object
Creates hierarchy	Avoids hierarchy
Less flexible for changing relationship	More flexible through interchangeable objects
Suited for closely related class types	Suited for combining independent responsibilities

Abstract Based Classes (ABC):

An Abstract Base Class (ABC) defines methods that must be implemented by its subclasses, ensuring that the subclasses follow a consistent structure. They allow to define common interfaces that various subclasses can implement while enforcing a level of abstraction.
Python provides the **'abc' module** to **define ABCs** and enforce the implementation of abstract methods in subclasses.

```
from abc import ABC, abstractmethod
class Animal(ABC):
    @abstractmethod
    def sound(self):
        pass

class Dog(Animal):
    def sound(self):
        return "Bark"

dog = Dog()
print(dog.sound())
```

Dunder (double underscore) methods:

Python magic (dunder) methods are special methods with double underscores "__" that enable custom object behaviour.

1. **__str__()** —> defines behaviour for when str() is called on an instance of the class.

```
s = 'Hello, Geeks.'
print (str(s))
print (str(2.0/11.0))
```

```
Hello, Geeks.
0.181818181818
```

2. **__repr__()** —> to get called by repr() method to return a machine readable representation of a type.

```
s = 'Hello, Geeks.'
print (repr(s))
print (repr(2.0/11.0))
```

Output:

```
'Hello, Geeks.'
0.1818181818181818
```

3. **__eq__()** —> defines behaviour for the "**==**" operator.

```
# lists initialization
x = [1, 2]
y = [1, 2]

z = y

# comparisons
print("x == y : ", x == y)
print("z == y : ", z == y)

# location in memory
print("Location of x is ", id(x))
print("Location of y is ", id(y))
print("Location of z is ", id(z))
```

Output

```
x == y :  True
z == y :  True
Location of x is  140169895643144
Location of y is  140169895642952
Location of z is  140169895642952
```

4. **__it__()** —> defines behaviour of the "**<**" operator.

```
class GFG:
    def __lt__(self, other):
        return "YES"
obj1 = GFG()
obj2 = GFG()

print(obj1 < obj2)
print(type(obj1 < obj2))
```

```
YES
<class 'str'>
```

Dataclasses:

The “dataclasses” module help define classes that mainly store data. It can automatically generate methods such as __init__(), __repr__(), __eq__().

```
from dataclasses import dataclass

@dataclass
class Employee:
    name: str
    emp_id: str
    age: int
    city: str

emp1 = Employee("Satyam", "ksatyam858", 21, "Patna")
emp2 = Employee("Anurag", "au23", 28, "Delhi")
emp3 = Employee("Satyam", "ksatyam858", 21, "Patna")

print(emp1)
print(emp2)
print(emp3)
print("emp1 and emp2 are same?", emp1 == emp2)
print("emp1 and emp3 are same?", emp1 == emp3)
```

Output

```
Employee(name='Satyam', emp_id='ksatyam858', age=21, city='Patna')
Employee(name='Anurag', emp_id='au23', age=28, city='Delhi')
Employee(name='Satyam', emp_id='ksatyam858', age=21, city='Patna')
emp1 ...
```

SOLID principles:

They are essential guidelines that enhance software design, making code more maintainable and scalable. They also help enhance loose coupling.
S —> single responsibility
O —> open-closed principle
L —> Liskov substitution principle
I —> interface segregation principle
D —> dependency inversion principle

Single responsibility principle:
It states that “a class should have only one reason to change”, meaning every class should have a single responsibility or single job.

Open-Closed principle:
It states that “software entities (classes, modules, etc.) should be open for extension, but closed for modification”, meaning should be able to extend class behaviour, without modifying it.

Liskov's Substitution Principle:
According to this principle, “derived or child classes must be able to replace their base or parent classes”. This ensures that any subclass can be used in place of its parent class without causing unexpected behavior in the program.

Interface Segregation principle:
It is similar to the Single Responsibility Principle. It states that, “clients should not be forced to depend on methods that are irrelevant to them, avoiding unnecessary dependencies.

Dependency inversion principle:
It states that “ High-level modules should not depend on low-level modules. Both should depend on abstractions”, meaning big parts of your program should not directly depend on small, detailed parts. Instead, both should depend on general ideas (interfaces).